`interface` or an `abstract class`, which can't be created directly.

Listing 9.3 shows how you can configure `EmailSender` and its dependencies in your application by using three methods: `AddScoped<TService>`, `AddSingleton<TService>`, and `AddScoped<TService, TImplementation>`. This code tells the DI container how to create each of the `TService` instances when they're required and which lifetime to use.

Listing 9.3 Registering services with the DI container

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.Services.AddScoped<IEmailSender, EmailSender>();    ❶
builder.Services.AddScoped<NetworkClient>();                ❷
builder.Services.AddSingleton<MessageFactory>();            ❸

WebApplication app = builder.Build();

app.MapGet("/register/{username}", RegisterUser);

app.Run();

string RegisterUser(string username, IEmailSender emailSender)
{
    emailSender.SendEmail(username);
    return $"Email sent to {username}!";
}
```

❶ Whenever you require an IEmailSender, use EmailSender.

❷ Whenever you require a NetworkClient, use NetworkClient.

❸ Whenever you require a MessageFactory, use MessageFactory.

That's all there is to DI! It may seem a little bit like magic, but you're simply giving the container instructions for making all the parts. You give it a recipe for cooking the chili, shredding the lettuce, and grating the cheese, so when